

Projet de data science – version 2, octobre 2026

Scoring de risque de crédit

Guide complet : des mathématiques à la mise en production

AUC du modèle retenu

0,923

Défauts détectés

76,4 %

Coût des erreurs vs v1

–37 %

Ce document explique tout le projet, pas à pas : le problème, les données, les statistiques et les algorithmes utilisés, la façon de les évaluer honnêtement, le code, les résultats, l'application et sa mise en ligne. Chaque notion est d'abord présentée par l'intuition, puis par sa formule, puis par son rôle concret dans le projet.

Mohamed Fofana et **Abdou Fall**

Projet de Compléments de mathématiques, L3 MIAHS, Université Grenoble Alpes — repris et amélioré en 2026

Application : <https://credit-risk-app-c8ye.onrender.com>

Code : <https://github.com/Mohamed-Fof/Credit-Risk>

Table des matières

1 Le problème : prêter sans se tromper	2
2 Les données	2
2.1 Source et variables	2
2.2 Nettoyage	3
2.3 Valeurs manquantes	3
2.4 Une découverte importante : des règles quasi déterministes	3
3 Le protocole : évaluer sans se mentir	3
3.1 Découpage apprentissage / test	3
3.2 Validation croisée à 5 plis	4
4 Les modèles : intuitions et mathématiques	4
4.1 Régression logistique	4
4.2 Logistique pénalisée (elastic net)	4
4.3 Naïve Bayes	5
4.4 Grille de score bancaire	5
4.5 Gradient boosting (XGBoost)	6
5 Mesurer la performance	6
5.1 AUC, Gini et KS	6
5.2 Calibration : score de Brier	7
5.3 Intervalles de confiance et test de DeLong	7
5.4 Du score à la décision : le seuil	7
6 Expliquer chaque décision : les valeurs de Shapley	7
7 Résultats	7
7.1 Tableau général	7
7.2 Pourquoi le modèle retenu n'a pas la meilleure AUC	8
7.3 La note du prêteur est-elle indispensable?	9
7.4 Ce qui compte pour le modèle	9
8 Le code : organisation et bonnes pratiques	9
8.1 Architecture	9
8.2 Une source unique pour la préparation	9
8.3 Chaque modèle comme un couple de fonctions	10
8.4 Tests automatiques	10
9 L'application : mode d'emploi	10

9.1 Utilisation	10
9.2 Contrôles et sécurité	12
10 La mise en production	13
10.1 Le conteneur Docker	13
10.2 Optimiser pour un serveur très modeste	13
10.3 Render et le cycle de mise à jour	14
11 Limites et pistes	14
12 Glossaire	15

1 Le problème : prêter sans se tromper

Une banque reçoit une demande de prêt. Elle doit décider de l'accorder ou non, sans savoir si l'emprunteur remboursera. Le *scoring de crédit* consiste à estimer, à partir des caractéristiques du dossier (revenu, montant demandé, historique...), la **probabilité de défaut** : la probabilité que l'emprunteur ne rembourse pas.

Les deux erreurs n'ont pas le même prix

Accorder un prêt à un futur défaillant (*faux négatif*) fait perdre une grande partie du capital prêté. Refuser un bon client (*faux positif*) ne fait perdre que la marge d'intérêt qu'il aurait rapportée. Dans ce projet, on suppose qu'un défaut accordé coûte **5 fois** plus qu'un refus à tort. Cette hypothèse guide tout le choix de la décision finale (section 5.4).

Un bon modèle de crédit doit réunir quatre qualités, et pas seulement la première :

1. **Discriminer** : donner un score plus risqué aux futurs défaillants qu'aux bons payeurs (mesuré par l'AUC).
2. **Être calibré** : quand il annonce 30 % de risque, il doit y avoir environ 30 % de défauts.
3. **Être cohérent** : le risque ne doit pas baisser quand on emprunte plus, ni monter quand le revenu augmente.
4. **Être explicable** : un conseiller doit pouvoir dire pourquoi un dossier est refusé.

Ce guide montre comment chacune de ces qualités a été construite et vérifiée, et pourquoi le modèle finalement retenu n'est pas celui qui a la meilleure AUC.

2 Les données

2.1 Source et variables

Le jeu public *Credit Risk Dataset* (Kaggle) contient 32 581 prêts à la consommation, avec leur issue : remboursé ou défaut. Les noms d'origine, en anglais, ont tous été traduits par un dictionnaire unique (app/preparation.R).

Nom dans le projet	Nom d'origine	Signification
age	person_age	âge de l'emprunteur
revenu_annuel	person_income	revenu annuel, en dollars
statut_logement	person_home_ownership	locataire, propriétaire, crédit immobilier en cours, autre
anciennete_emploi	person_emp_length	ancienneté professionnelle, en années
motif_pret	loan_intent	études, santé, création d'entreprise, personnel, travaux, regroupement de dettes
note_risque	loan_grade	note de A (meilleure) à G attribuée par le prêteur
montant_pret	loan_amnt	montant emprunté
taux_interet	loan_int_rate	taux d'intérêt, en %
taux_effort	recalculé	montant / revenu
defaut_anterieur	cb_person_default_on_file	défait de paiement déjà enregistré (oui/non)
anciennete_credit	cb_person_cred_hist_length	ancienneté de l'historique de crédit
defaut	loan_status	cible : 1 = défaut, 0 = remboursé

2.2 Nettoyage

- **165 doublons exacts** sont retirés *avant* tout découpage. Sinon, un même dossier pourrait se retrouver à la fois dans les données d'apprentissage et de test : le modèle serait évalué sur un dossier qu'il a déjà vu (une *fuite d'information*).
- 5 âges supérieurs à 100 ans et 2 anciennetés professionnelles impossibles (jusqu'à 123 ans) sont retirés.
- Le taux d'effort fourni était arrondi à deux décimales : il est recalculé exactement.

Il reste **32 409 dossiers**, dont 21,9 % de défauts.

2.3 Valeurs manquantes

Le taux d'intérêt manque pour 9,5 % des dossiers et l'ancienneté pour 2,7 %. Ces valeurs sont remplacées par la *médiane*, calculée sur les seules données d'apprentissage. Mais l'absence elle-même est une information : un dossier où le taux n'est pas renseigné n'est pas un dossier quelconque. On ajoute donc deux indicatrices, `taux_manquant` et `anciennete_manquante`, qui valent 1 quand la valeur était absente.

2.4 Une découverte importante : des règles quasi déterministes

En étudiant le taux de défaut selon le taux d'effort, on découvre une rupture brutale : chez les locataires notés A ou B dont le taux d'effort dépasse 32 %, le taux de défaut est de **100,0 %** sur 1 154 dossiers.

Limite majeure du jeu de données

Aucun portefeuille de crédit réel ne présente un seuil aussi parfait. Ce jeu de données est en partie simulé : il contient des règles qu'un modèle flexible apprend presque par cœur. Les performances obtenues ici sont donc *optimistes* et ne se transposeraient pas telles quelles à une banque. C'est la première chose à dire en entretien quand on présente ce projet.

3 Le protocole : évaluer sans se mentir

L'erreur la plus fréquente en data science n'est pas un mauvais modèle, c'est une *évaluation trop optimiste*. Le protocole sert à garantir que les chiffres annoncés valent pour des dossiers que le modèle n'a jamais vus.

3.1 Découpage apprentissage / test

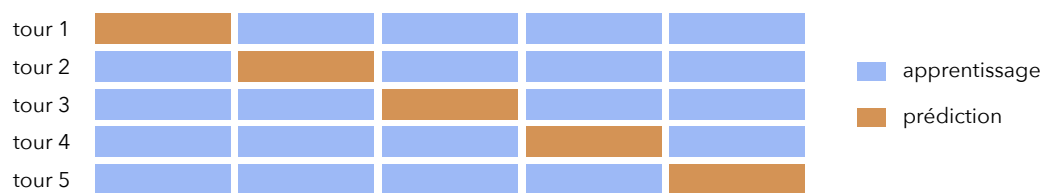
Les données sont séparées en deux, *avant toute autre transformation* :

- **apprentissage** (25 927 dossiers, 80 %) : sert à tout — régler, entraîner, choisir le seuil ;
- **test** (6 482 dossiers, 20 %) : mis sous clé, utilisé *une seule fois*, à la toute fin.

Le découpage est **stratifié** : le taux de défaut est le même des deux côtés (21,9 %). Sans cela, un tirage malheureux pourrait mettre plus de défauts d'un côté et fausser la comparaison.

3.2 Validation croisée à 5 plis

Pour régler les modèles sans toucher au test, l'échantillon d'apprentissage est découpé en 5 parts (*plis*) stratifiées. On entraîne sur 4 plis et on prédit le cinquième, cinq fois de suite. Chaque dossier d'apprentissage reçoit ainsi une prédiction *hors pli*, faite par un modèle qui ne l'a pas vu.



Deux précautions rendent ce protocole rigoureux :

- les **mêmes plis** servent à tous les modèles : la comparaison est équitable ;
- la **préparation est réapprise dans chaque pli** : la médiane utilisée pour imputer est calculée sur les 4 plis d'entraînement, jamais sur le pli prédit.

Toutes les étapes aléatoires utilisent la graine 2026 : relancer le projet redonne exactement les mêmes chiffres.

4 Les modèles : intuitions et mathématiques

Six modèles sont comparés, du plus simple au plus flexible. On note $x \in \mathbb{R}^p$ les caractéristiques d'un dossier et $Y \in \{0, 1\}$ le défaut.

4.1 Régression logistique

Intuition

On additionne les effets des variables sur une échelle « log-cote », puis on transforme ce total en probabilité comprise entre 0 et 1.

Le modèle s'écrit

$$\mathbb{P}(Y = 1 \mid x) = \sigma(\beta_0 + \beta^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

La quantité $\log \frac{p}{1-p}$ est la *log-cote* (log-odds) : elle est linéaire en x . Ainsi e^{β_j} est le *rapport de cotes* : de combien la cote de défaut est multipliée quand x_j augmente d'une unité. Les coefficients maximisent la log-vraisemblance

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)],$$

par l'algorithme de Newton-Raphson (moindres carrés repondérés itérés). Dans le projet, le revenu et le montant entrent en logarithme, car leur effet est proportionnel plutôt qu'additif, et les variables qualitatives sont codées en indicatrices.

4.2 Logistique pénalisée (elastic net)

Pour éviter le sur-apprentissage et sélectionner les variables utiles, on ajoute une pénalité :

$$\hat{\beta} = \arg \min_{\beta} -\frac{1}{n} \ell(\beta) + \lambda \left[\frac{1 - \alpha}{2} \|\beta\|_2^2 + \alpha \|\beta\|_1 \right].$$

La partie $\|\beta\|_1$ (lasso) peut annuler des coefficients ; la partie $\|\beta\|_2^2$ (ridge) stabilise les variables corrélées. α et λ sont choisis par validation croisée (ici $\alpha = 0,25$). Résultat : même performance que la logistique simple (0,876), ce qui montre que la logistique ne sur-apprenait pas — sa limite est sa *forme* linéaire, pas son nombre de paramètres.

4.3 Naïve Bayes

Par la formule de Bayes, et en supposant les variables indépendantes conditionnellement à la classe :

$$\mathbb{P}(Y = k | x) \propto \mathbb{P}(Y = k) \prod_{j=1}^p f_j(x_j | Y = k).$$

L'hypothèse d'indépendance est fautive ici (le taux d'effort dépend du montant et du revenu), d'où des probabilités moins bien calibrées. Pour les variables numériques, la densité f_j est estimée par **noyau** plutôt que par une loi normale :

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right),$$

ce qui épouse les distributions asymétriques (revenu, montant). Gain mesuré en validation croisée : AUC 0,872 contre 0,847 avec une loi normale.

4.4 Grille de score bancaire

C'est la méthode historique des banques, appréciée parce qu'un conseiller peut la lire.

1. **Discretisation** : chaque variable est découpée en classes (par exemple, taux d'effort $< 10\%$, $10-20\%$...), choisies par un arbre de décision.
2. **Poids de l'évidence (WoE)** de la classe i :

$$\text{WoE}_i = \ln \frac{D_i/D}{N_i/N},$$

où D_i et N_i comptent les défauts et non-défauts de la classe, D et N les totaux. Un WoE positif signale une classe plus risquée que la moyenne.

3. **Valeur d'information** d'une variable : $IV = \sum_i \left(\frac{D_i}{D} - \frac{N_i}{N}\right) \text{WoE}_i$. On ne garde que les variables d'IV supérieure à 0,02.
4. **Régression logistique sur les WoE**, puis conversion en points :

$$\text{score} = A - B \ln(\text{cote}), \quad B = \frac{\text{PDO}}{\ln 2}, \quad A = 600 + B \ln \frac{1}{19}.$$

600 points correspondent à une cote de 1 contre 19 (5 % de risque), et chaque tranche de 50 points (PDO, *points to double the odds*) divise la cote par deux. Chaque classe reçoit $-B \beta_j \text{WoE}_{ij}$ points.

Règle des signes

Tous les coefficients de la logistique sur WoE doivent être positifs. Ici, le défaut antérieur obtenait un coefficient négatif, car son information est déjà contenue dans la note du prêteur (colinéarité) : un défaut antérieur *rapportait* des points. La variable a été retirée et le modèle réestimé, comme le veut la pratique bancaire. Un test automatique vérifie aussi qu'un taux d'effort plus élevé ne rapporte jamais de points.

4.5 Gradient boosting (XGBoost)

Intuition

On construit une suite de petits arbres de décision. Chaque nouvel arbre corrige les erreurs de la somme des précédents. La prédiction est la somme de tous les arbres.

Le modèle est additif, sur l'échelle log-cote : $F(x) = \sum_{m=1}^M f_m(x)$, où chaque f_m est un arbre. On minimise

$$\mathcal{L} = \sum_i l(y_i, F(x_i)) + \sum_m \Omega(f_m), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2,$$

où l est la perte logistique, T le nombre de feuilles et w leurs valeurs. À l'étape m , XGBoost approche la perte par un développement de Taylor d'ordre 2, avec g_i et h_i les dérivées première et seconde de l en $F_{m-1}(x_i)$. Pour une feuille contenant les dossiers I_j , la valeur optimale et le gain d'une coupure valent

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \text{gain} = \frac{1}{2} \left[\frac{G_G^2}{H_G + \lambda} + \frac{G_D^2}{H_D + \lambda} - \frac{(G_G + G_D)^2}{H_G + H_D + \lambda} \right] - \gamma.$$

Chaque arbre est ajouté avec un *taux d'apprentissage* $\eta = 0,05$: $F_m = F_{m-1} + \eta f_m$. On sous-échantillonne 80 % des dossiers et des variables à chaque arbre, et le nombre d'arbres est fixé par **arrêt précoce** : on s'arrête quand l'AUC de validation croisée ne progresse plus depuis 100 arbres.

Contraintes de monotonie

On peut imposer qu'une prédiction soit monotone en une variable : XGBoost refuse alors toute coupure dont les deux branches violeraient l'ordre, et propage des bornes aux sous-arbres. Le modèle retenu impose que, toutes choses égales par ailleurs, le risque *croisse* avec le montant, le taux d'effort, le taux d'intérêt, la note et un défaut antérieur, et *décroisse* avec le revenu. La section 7.2 explique pourquoi c'est indispensable.

Réglage retenu (validation croisée, 12 combinaisons) : profondeur 7, poids minimal par feuille 5, 184 arbres.

5 Mesurer la performance

5.1 AUC, Gini et KS

L'**AUC** est l'aire sous la courbe ROC. Elle a une interprétation très concrète : c'est la probabilité qu'un futur défaillant tiré au hasard reçoive un score plus risqué qu'un bon payeur tiré au hasard. On la calcule exactement par la statistique de Mann-Whitney :

$$\text{AUC} = \frac{R_1 - n_1(n_1 + 1)/2}{n_1 n_0},$$

où R_1 est la somme des rangs des scores des défauts, n_1 et n_0 les effectifs. 0,5 correspond au hasard, 1 à un classement parfait.

Les banques utilisent le **Gini** = $2 \text{AUC} - 1$ et le **KS**, écart maximal entre les fonctions de répartition des scores des deux populations : $\text{KS} = \max_s |F_1(s) - F_0(s)|$.

5.2 Calibration : score de Brier

LAUC ne regarde que l'ordre des scores, pas leur valeur. Le **score de Brier**, $\frac{1}{n} \sum_i (p_i - y_i)^2$, mesure si les probabilités sont justes. La courbe de calibration compare, par décile, la probabilité moyenne annoncée au taux de défaut réellement observé.

5.3 Intervalles de confiance et test de DeLong

LAUC mesurée sur 6 482 dossiers est une estimation : la méthode de DeLong fournit sa variance, donc un intervalle de confiance à 95 %. Elle permet aussi de tester si deux modèles diffèrent vraiment sur les mêmes dossiers (test apparié). Tous les écarts entre le modèle retenu et les modèles linéaires sont significatifs ($p < 0,0001$).

5.4 Du score à la décision : le seuil

Le modèle donne une probabilité p ; il faut décider : refuser si $p \geq s$. Le seuil $s = 0,5$ de la version 1 laissait passer 41 % des défauts. On choisit plutôt le seuil qui minimise le coût moyen

$$C(s) = \frac{1}{n} [5 \cdot \text{FN}(s) + 1 \cdot \text{FP}(s)].$$

Le seuil théorique

Pour un modèle parfaitement calibré, refuser coûte $(1 - p) \cdot 1$ (on perd un bon client avec probabilité $1 - p$) et accepter coûte $p \cdot 5$. On refuse donc dès que $5p > 1 - p$, soit $p > 1/6 \approx 16,7\%$. Le seuil réellement retenu (25,5 %) est cherché numériquement sur les prédictions hors pli, *jamais* sur le test, pour tenir compte des imperfections de calibration.

6 Expliquer chaque décision : les valeurs de Shapley

Un modèle à plusieurs centaines d'arbres n'est pas lisible directement. Les **valeurs de Shapley**, issues de la théorie des jeux coopératifs, répartissent la prédiction entre les variables de façon équitable. La contribution de la variable j est sa contribution marginale moyenne sur tous les ordres d'arrivée possibles des variables :

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} [v(S \cup \{j\}) - v(S)].$$

Propriété clé (*efficacité*) : la somme des contributions, plus une valeur de base, redonne exactement la prédiction en log-cote,

$$F(x) = \phi_0 + \sum_j \phi_j.$$

Pour les arbres, l'algorithme TreeSHAP calcule ces valeurs exactement et rapidement. Dans l'application, chaque dossier affiche ses contributions : en rouge ce qui augmente le risque, en vert ce qui le réduit. Un test automatique vérifie la propriété d'efficacité.

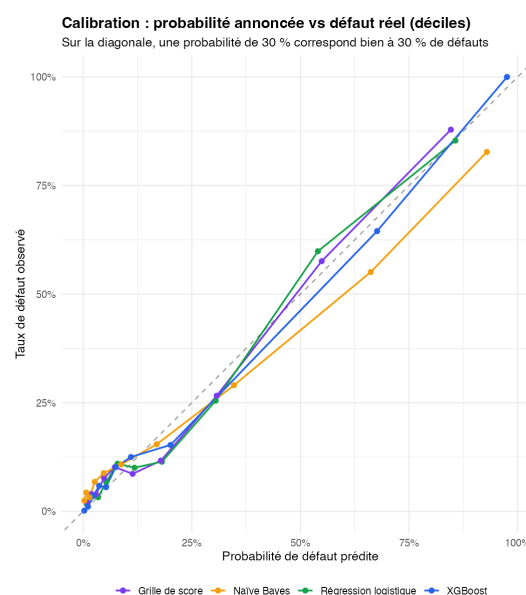
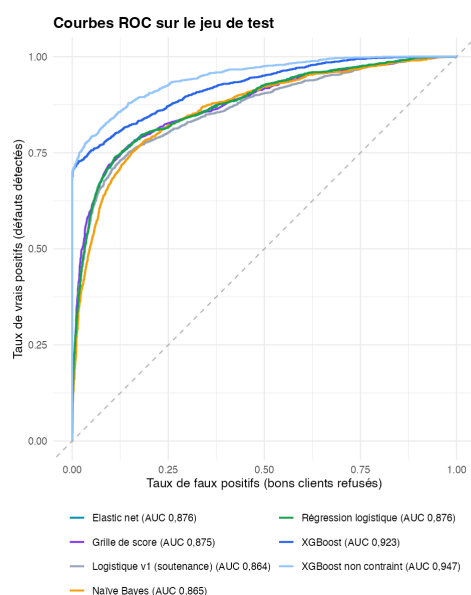
7 Résultats

7.1 Tableau général

Mesures sur les 6 482 dossiers de test, jamais utilisés auparavant.

Modèle	AUC	IC 95 %	Défauts détectés	Refus justifiés	Coût
XGBoost contraint (retenu)	0,923	[0,914 ; 0,932]	76,4 %	76,4 %	0,310
XGBoost non contraint	0,947	[0,940 ; 0,954]	83,4 %	—	0,258
Régression logistique	0,876				
Elastic net	0,876				
Grille de score	0,875				
Naïve Bayes	0,865				
<i>Logistique v1 (soutenance)</i>	<i>0,864</i>		<i>58,8 %</i>	<i>77,2 %</i>	<i>0,489</i>

Le tableau complet (Gini, KS, Brier, spécificité...) est dans `resultats/metriques_test.csv` et dans l'onglet « Comparaison des modèles » de l'application.



7.2 Pourquoi le modèle retenu n'a pas la meilleure AUC

Sans contrainte, XGBoost atteint 0,947. Mais un audit de cohérence change tout : on modifie *une seule* caractéristique de chaque dossier de test et on regarde si le risque bouge dans le bon sens.

Dossiers à contresens (écart > 1 point)	Montant +20 %	Revenu +20 %	Taux +2 points
XGBoost non contraint	18,6 %	23,0 %	10,0 %
Régression logistique	4,1 %	0 %	0 %
XGBoost contraint (retenu)	0 %	0 %	0 %

Sans contrainte, le modèle juge un emprunteur *plus risqué quand son revenu augmente* pour 23,0 % des dossiers : il a appris le bruit des données (le taux de défaut observé n'est pas monotone en revenu, par simple hasard d'échantillonnage). Un tel modèle n'est ni crédible pour un utilisateur, ni défendable devant un régulateur. Les contraintes coûtent 0,024 d'AUC (de 0,947 à 0,923) : c'est le **prix de la cohérence**, et il est assumé.

Une variante a été testée : retirer le revenu brut (il reste présent via le taux d'effort) pour garantir la cohérence sans contrainte sur le revenu. L'AUC tombe alors à 0,884 : le revenu apporte une information propre, la contrainte est la bonne solution.

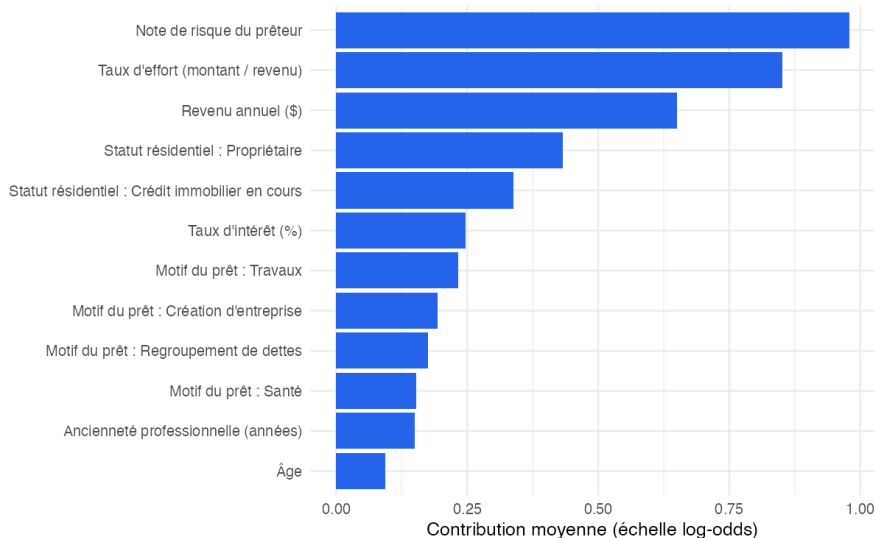
7.3 La note du prêteur est-elle indispensable ?

La note A à G est attribuée par le prêteur lui-même : l'utiliser revient en partie à recopier son avis. Sans elle, le modèle retenu atteint encore une AUC de **0,913** : il apprend réellement des caractéristiques de l'emprunteur et du prêt.

7.4 Ce qui compte pour le modèle

Variables les plus influentes (XGBoost)

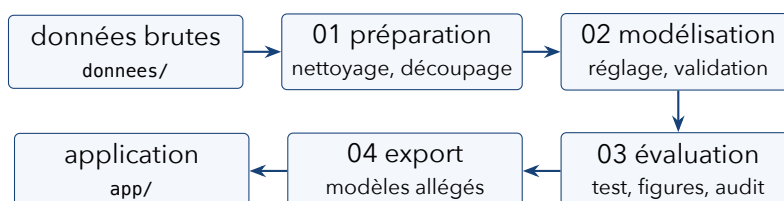
Contribution moyenne à la décision (valeurs de Shapley, en valeur absolue)



La note du prêteur, le taux d'effort et le revenu dominant, suivis du statut résidentiel : les propriétaires font beaucoup moins défaut que les locataires.

8 Le code : organisation et bonnes pratiques

8.1 Architecture



Une seule commande reproduit tout, en trois minutes environ : `Rscript R/executer_tout.R`.

8.2 Une source unique pour la préparation

Le piège classique de la mise en production est l'écart entre la préparation faite à l'entraînement et celle faite dans l'application. Ici, `app/preparation.R` est utilisé *par les deux* :

```

appliquer_preparation <- function(df, params) {
  # L'absence d'information est conservée comme information
  df$taux_manquant <- as.integer(is.na(df$taux_interet))
  df$taux_interet[is.na(df$taux_interet)] <- params$mediane_taux
}

```

```
df$taux_effort <- df$montant_pret / df$revenu_annuel
...
}
```

L'export vérifie ensuite que l'application reproduit *exactement* (à 10^{-6} près) les prédictions de l'entraînement sur les 6 482 dossiers de test.

8.3 Chaque modèle comme un couple de fonctions

Tous les modèles se décrivent de la même façon : une fonction ajuster et une fonction prédire. La validation croisée devient alors générique, et la préparation est réapprise dans chaque pli :

```
hors_pli <- function(modele) {
  p <- numeric(nrow(apprentissage))
  for (k in 1:5) {
    m <- modele$ajuster(apprentissage[plis != k, ])
    p[plis == k] <- modele$predire(m, apprentissage[plis == k, ])
  }
  p
}
```

8.4 Tests automatiques

tests/tests.R contient 26 tests, à relancer avant chaque mise en ligne : contrôle des saisies (y compris malveillantes), préparation, reproduction exacte des prédictions, additivité des valeurs de Shapley, cohérence du modèle sur *tous* les dossiers de test, cohérence de la grille de score.

9 L'application : mode d'emploi

9.1 Utilisation

L'application compte quatre onglets.

- **Simulateur** : on saisit un dossier à gauche ; l'application affiche la décision recommandée, la probabilité de défaut, l'explication de la décision et le score de la grille bancaire détaillé point par point, ainsi que l'avis des cinq modèles. Le calcul se relance automatiquement 0,4 seconde après la dernière saisie.
- **Comparaison des modèles** : performances, courbes ROC, calibration, importance des variables, audit de cohérence.
- **Exploration des données** : distributions et taux de défaut par modalité.
- **Méthode** : protocole, hypothèses et limites.

Scoring de risque de crédit

Simulateur

Comparaison des modèles

Exploration des données

Méthode

Code source

Dossier de l'emprunteur

Âge

30

Revenu annuel (\$)

50000

Statut résidentiel

Locataire

Ancienneté professionnelle (années)

5

Ancienneté de l'historique de crédit (années)

4

Défaut de paiement antérieur

Non

Montant du prêt (\$)

10000

Motif du prêt

Personnel

Taux d'intérêt (%)

11

Note de risque attribuée par le prêteur

B

A = meilleure note, G = la plus risquée.

DÉCISION RECOMMANDÉE

Prêt envisageable

Seuil de refus : 25,5 % de probabilité de défaut

PROBABILITÉ DE DÉFAUT

5,1 %

MODÈLE

XGBoost

le plus performant sur le test

Pourquoi cette décision ?

Taux d'effort (montant / revenu)

Note de risque du prêteur

Statut résidentiel

Revenu annuel (\$)

Ancienneté professionnelle (années)

Motif du prêt

Âge

Taux d'intérêt (%)

-0,6

-0,3

0

0,3

Contribution de chaque caractéristique à la probabilité de défaut estimée par XGBoost (valeurs de Shapley). En rouge : ce qui augmente le risque ; en vert : ce qui le réduit.

Grille de score bancaire

516 points

soit 14,4 % de risque - avis : accord

Méthode des banques : chaque caractéristique rapporte ou retire des points. 600 points correspondent à 5 % de risque ; 50 points de plus divisent la cote de risque par deux. C'est un modèle distinct de XGBoost, plus simple et entièrement lisible.

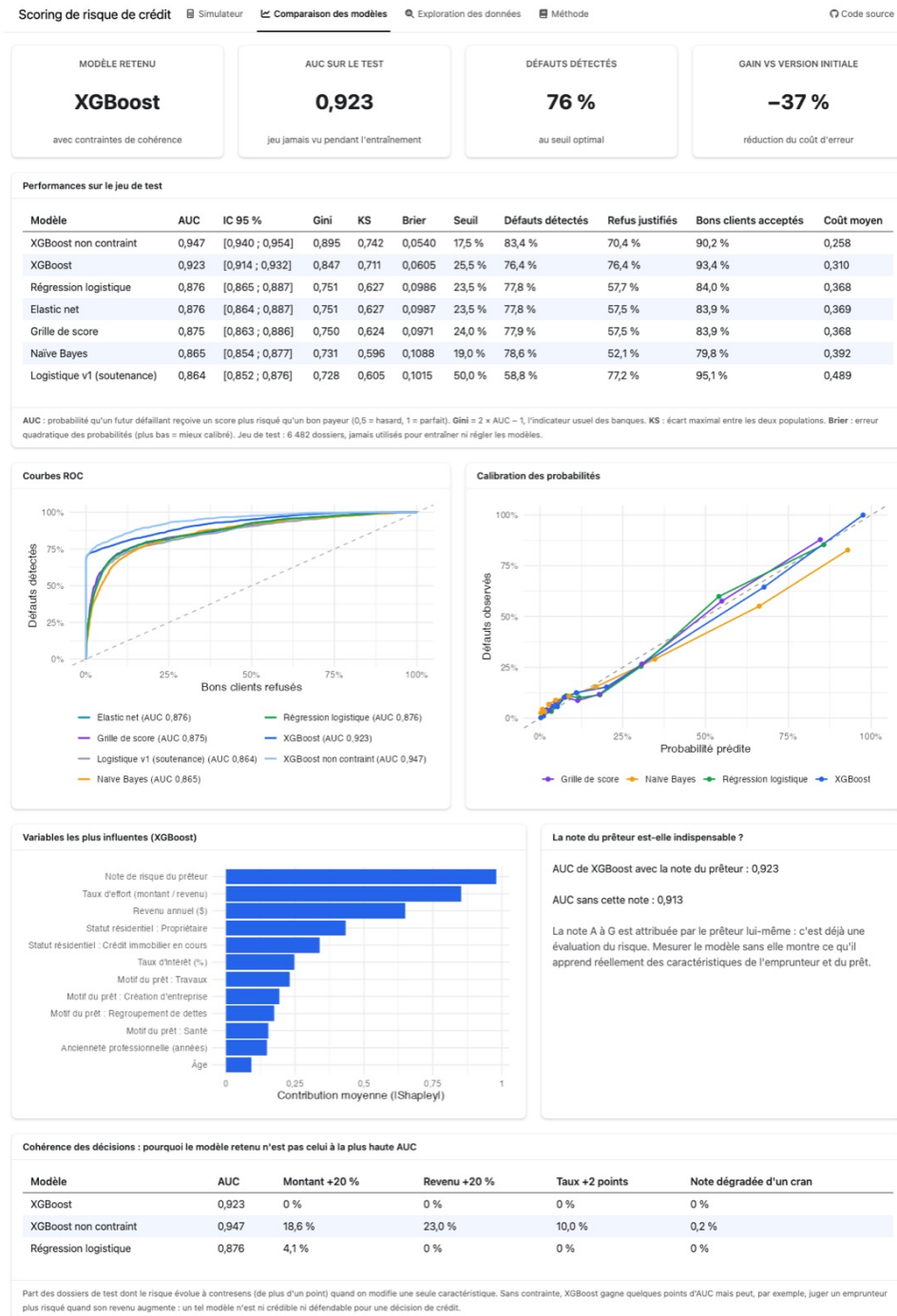
Caractéristique	Points
Statut résidentiel	-34
Note de risque du prêteur	+28
Motif du prêt	+12
Revenu annuel (\$)	+8
Taux d'effort (montant / revenu)	+7
Ancienneté professionnelle (années)	+4
Montant du prêt (\$)	+3
Taux d'intérêt (%)	+2

Avis des cinq modèles

Modèle	Probabilité de défaut	Seuil du modèle	Avis
XGBoost	5,1 %	25,5 %	Accord
Grille de score	14,4 %	24,0 %	Accord
Régression logistique	13,5 %	23,5 %	Accord
Elastic net	13,7 %	23,5 %	Accord
Naïve Bayes	5,9 %	19,0 %	Accord

Chaque seuil minimise le coût attendu en validation croisée, avec l'hypothèse qu'accorder un prêt à un futur défaillant coûte 5 fois plus que refuser un bon client.

Le simulateur : décision, explication par les valeurs de Shapley, grille de score, avis des modèles.



Longlet de comparaison des modèles.

9.2 Contrôles et sécurité

- **Saisies revérifiées côté serveur.** Shiny ne contrôle pas que la valeur d'une liste déroulante fait partie des choix proposés : un client modifié peut envoyer n'importe quoi. Chaque valeur est donc revérifiée (liste blanche, type, bornes, cohérence âge / ancienneté / historique, montant / revenu). Cinq attaques ont été simulées depuis un navigateur modifié (HTML piégé, code R, valeur vide, tableau, nombre géant) : toutes sont bloquées.
- **Extrapolation signalée** : une valeur hors de l'intervalle observé à l'entraînement déclenche un avertissement.
- **Erreurs masquées** : l'utilisateur ne voit jamais de message technique (option shiny::sanitize).

errors).

- **Aucune donnée conservée** : les dossiers saisis ne sont ni enregistrés ni transmis.

10 La mise en production

10.1 Le conteneur Docker

L'application est empaquetée dans une image Docker, une « boîte » qui contient R, les paquets et le code, et qui tourne à l'identique partout.

```
FROM rocker/r-ver:4.5.1                # R minimal (365 Mo)
RUN R -e "install.packages(c('shiny','bslib','ggplot2','xgboost','naivebayes'))"
COPY app/ /app/                        # 2,3 Mo de modèles
USER appli                             # pas de droits administrateur
CMD Rscript -e "shiny::runApp('/app', host='0.0.0.0', port=...)"
```

Choix faits pour démarrer vite :

- image minimale (365 Mo contre 580 Mo pour l'image Shiny complète de la version 1) ;
- paquets *précompilés* (Posit Package Manager), figés à la date d'entraînement : la version de XGBoost qui lit le modèle est celle qui l'a créé ;
- modèles allégés : 2,3 Mo chargés une fois au démarrage, contre un modèle de 40 Mo rechargé à chaque prédiction dans la version 1. L'application est prête en moins de 2 secondes une fois le conteneur lancé ;
- **contrôle à la construction** : l'image vérifie que l'application se charge et que sa page d'accueil s'affiche avant d'être produite ; GitHub Actions refait ce test, conteneur lancé, à chaque envoi de code. Si quelque chose casse, la construction échoue et Render garde la version précédente en ligne au lieu d'afficher une erreur aux visiteurs.

Retour d'expérience : la locale

Lors du premier déploiement, l'application fonctionnait parfaitement en local mais affichait une erreur en ligne. La cause : le conteneur Linux ne déclarait pas de locale UTF-8, et R ne lisait plus les accents des scripts (« Régression », « Naïve Bayes »). Le diagnostic a consisté à reproduire localement la différence (LANG=C), ce qui a fait apparaître exactement la même erreur. Mais ce n'était qu'une partie du problème : une fois la locale corrigée, l'intégration continue (GitHub Actions, qui construit et lance l'image à chaque envoi) a révélé une seconde cause, la bibliothèque système `libuv` manquante pour le paquet `fs`, utilisé à la compilation du thème. Les paquets précompilés n'embarquent pas leurs dépendances système. Corrections : `ENV LANG=C.UTF-8`, installation de `libuv1`, et un contrôle à la construction qui affiche réellement la page d'accueil. Leçon : un contrôle doit reproduire l'usage réel (ici, afficher la page), sinon il laisse passer des erreurs.

10.2 Optimiser pour un serveur très modeste

L'offre gratuite de Render n'accorde qu'environ 0,1 processeur : chaque calcul y est 10 à 15 fois plus lent que sur un ordinateur portable. Une fois en ligne, l'application mettait 5 à 8 secondes à afficher sa première décision. Plutôt que de deviner, chaque étape a été *mesurée*, puis le serveur a été *profilé* (`Rprof`) pendant que des navigateurs automatisés enchaînaient les saisies.

- **Les modèles n'étaient pas en cause** : les cinq prédictions, valeurs de Shapley comprises, prennent 7 ms.
- **La page était reconstruite à chaque visite** : 65 % de ce temps servait à recompiler le thème graphique (Sass). Elle est désormais construite *une fois* au démarrage, puis resservie : 223 ms → 1,3 ms par visite.
- **Un composant de mise en page recompilait ses styles à chaque saisie** (`layout_columns` dans un élément redessiné) : remplacé par une grille CSS.
- **Les graphiques sont dessinés à l'avance** (onglets Comparaison et Exploration) et l'explication devient des barres HTML : `ggplot2` n'est plus chargé.
- **La décision du dossier par défaut est incluse dans la page** : le visiteur voit un résultat complet avant même que la connexion au serveur soit établie.

Mesure en production	Avant	Après
Réponse du serveur (premier octet)	2,0 à 3,5 s	0,27 s
Page affichée avec sa décision	5,5 à 8 s	0,5 à 0,7 s
Nouvelle décision après une saisie (dont 0,3 s d'attente volontaire)	1,1 s	0,9 s

10.3 Render et le cycle de mise à jour

Render (offre gratuite) surveille le dépôt GitHub. À chaque `git push` sur la branche principale : construction de l'image à partir du `Dockerfile`, lancement du conteneur, bascule du trafic. Mettre à jour l'application, c'est donc : modifier le code, relancer `R/executer_tout.R` et `tests/tests.R`, puis pousser.

Le réveil de l'offre gratuite

Render endort le service après 15 minutes sans visite, et le réveil prenait 72 secondes : un recruteur aurait attendu plus d'une minute. Trois parades se complètent : le portfolio contacte l'application en arrière-plan dès qu'un visiteur l'ouvre ; une tâche GitHub Actions (`.github/workflows/reveil-render.yml`) l'appelle toutes les 10 minutes, mais GitHub ne garantit pas l'exécution des tâches planifiées ; un service de surveillance gratuit (UptimeRobot, cron-job.org) est la solution la plus fiable. Contrepartie : le service tourne en continu, ce qui consomme les 750 heures gratuites mensuelles de Render.

11 Limites et pistes

- **Données en partie simulées** (section 2.4) : les performances sont optimistes. Sur des données réelles, une AUC de 0,75 à 0,85 serait déjà bonne.
- **Pas de dimension temporelle** : impossible de vérifier la stabilité du modèle dans le temps. En banque, on validerait sur une période postérieure (*out-of-time*) et on surveillerait la dérive des données en production (indice de stabilité de population, PSI).
- **Hypothèse de coût** : le ratio 5 pour 1 devrait être calibré sur les pertes réelles (probabilité de défaut × perte en cas de défaut).
- **Équité** : l'âge est utilisé, alors que c'est un critère sensible. Une suite naturelle serait de mesurer l'écart de décisions entre groupes d'âge et l'impact de son retrait.
- **Usage** : l'application est un outil pédagogique, pas une décision de crédit.

12 Glossaire

AUC

probabilité qu'un défaillant reçoive un score plus risqué qu'un bon payeur.

Calibration

accord entre probabilités annoncées et fréquences observées.

Contrainte de monotonie

obligation pour le modèle de faire varier le risque toujours dans le même sens avec une variable.

Fuite d'information

information du jeu de test qui s'infiltre dans l'entraînement et rend l'évaluation trop optimiste.

Hors pli

prédiction faite par un modèle qui n'a pas vu le dossier.

Log-cote

$\log \frac{p}{1-p}$, échelle sur laquelle les effets s'additionnent.

Valeurs de Shapley

répartition équitable d'une prédiction entre les variables.

WoE / IV

poids de l'évidence d'une classe et pouvoir discriminant d'une variable, outils des grilles de score.

Reproduire ce document : `Rscript R/executer_tout.R`, puis `Rscript R/05_chiffres_document.R`, puis `xelatex guide_projet.tex` dans docs/. Tous les chiffres sont recalculés à partir des résultats.